

CS 650 – Full Stack Application Develop

10-1 Short Paper: Full Stack Roles, Tools, and Security

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

September 8, 2026

Reflection on Full Stack Application Development

Front-End and Back-End Development

Front-end and back-end development contribute to the same user experience but require different priorities. The front end must make behavior understandable, responsive, and accessible. It demands careful management of component state, navigation, validation feedback, and recovery from errors. Back-end development emphasizes data integrity, authorization, business rules, API contracts, and dependable error handling. The server must treat requests as untrusted, verify permissions, and preserve correct data when a client sends unexpected input.

I found back-end development most engaging because it combines programming logic with security and reliability. In the recipe service work, I enjoyed tracing how normalized input, database queries, and matching rules produced the response used by the interface. A single server-side authorization decision can also protect every client. That responsibility connects to my goal of building technology for sensitive human-services environments, where confidentiality is essential.

Tools and Technologies

The course stack worked well because TypeScript made data structures and API expectations explicit while still supporting rapid development. Node-based server code reduced language switching, component-based front-end code divided interface behavior into manageable pieces, and automated tests repeatedly checked functions and request paths. GitHub version control and continuous integration also modeled a professional workflow in which a proposed change could be reviewed and verified before release.

Next.js might have been a better fit for a content-heavy public application needing server rendering, search engine visibility, and one deployment model. It can combine interface rendering and server routes and reduce setup. For this course, however, a separated stack made API, authentication, and deployment boundaries visible. For a sensitive transactional application, I would continue to favor clearly separated services and explicit authorization controls.

Security Practices

The security exercises showed that vulnerabilities are often exploited through ordinary application behavior. In the insecure direct object reference exercise, changing an account identifier in the URL requested another user's information because the server did not verify ownership. Hiding identifiers or disabling a button is not accessing control; authorization must be enforced on the server for every protected resource. OWASP warns that predictable identifiers become dangerous when an application fails to validate the requester's authority (OWASP Foundation, 2021a).

The petition application monitors recorded authentication events, unauthorized attempts, and signature activity. These events could reveal repeated failed logins, access attempts across multiple identifiers, or unusual request bursts, especially if centralized logs triggered alerts. Monitoring would not prevent IDOR, however, and isolated events might appear harmless without correlation. It must complement authorization, secure sessions, and input validation. OWASP emphasizes security-event monitoring, while NIST recommends integrating secure practices throughout the development life cycle (NIST, 2022; OWASP Foundation, 2021b).

Practices That Support Application Quality

Testing, CI/CD, monitoring, and release management form a feedback system. Tests check expected behavior; the CI pipeline repeats those checks consistently and blocks known failures. Release management identifies the approved version, deployment and verification of steps, responsibilities, and rollback plan. After deployment, monitoring supplies evidence about behavior that pre-release tests cannot reproduce. Production findings then become defects, new tests, and release-plan improvements. Weakness in any practice creates a gap. Without meaningful tests, CI can pass unverified behavior; without CI, tests may be skipped. Without monitoring, attacks or failures may remain invisible. Without release controls, a tested build can be deployed with the wrong configuration or no recovery path. The release runbook reinforced that quality is a chain of evidence and safeguards, not one final inspection.

Design Decision I Would Change

I would establish centralized authorization and structured security logging at the beginning of a project rather than after feature development. Reusable middleware or service functions would authenticate the user, verify permission for the requested resource, and record a consistent event with the user or session, action, resource, result, timestamp, and correlation identifier. Tests would cover allowed and denied cases, and CI would require them to pass.

This change would prevent developers from recreating checks in individual routes, give reviewers one consistent pattern to evaluate, and allow events from different features to be correlated. Secure behavior would become part of the definition of done. The course showed me that reliability is strongest when architecture, testing, automation, monitoring, and release planning are designed as one system.

References

National Institute of Standards and Technology. (2022). *Secure software development framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities* (NIST Special Publication 800-218).

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-218.pdf>

OWASP Foundation. (2021a). *A01:2021—Broken access control*.

https://owasp.org/Top10/2021/A01_2021-Broken_Access_Control/

OWASP Foundation. (2021b). *A09:2021—Security logging and monitoring failures*.

https://owasp.org/Top10/2021/A09_2021-Security_Logging_and_Monitoring_Failures/